arXiv:2506.06975v3 [cs.CR] 11 Jun 2025

Auditing Black-Box LLM APIs with a Rank-Based Uniformity Test

Xiaoyuan Zhu[1] **Yaowen Ye**[2†] **Tianyi Qiu**[3†] **Hanlin Zhu**[2‡] **Sijun Tan**[2‡]
Ajraf Mannan[1] **Jonathan Michala**[1] **Raluca Ada Popa**[2] **Willie Neiswanger**[1]
[1]University of Southern California [2]University of California, Berkeley [3]Peking University
xzhu9839@usc.edu, elwin@berkeley.edu, qiutianyi.qty@gmail.com
{hanlinzhu, sijuntan}@berkeley.edu, {amannan, michala}@usc.edu
raluca@eecs.berkeley.edu, neiswang@usc.edu
† Co-second authors. ‡ Co-third authors.

Abstract

As API access becomes a primary interface to large language models (LLMs), users often interact with black-box systems that offer little transparency into the deployed model. To reduce costs or maliciously alter model behaviors, API providers may discreetly serve quantized or fine-tuned variants, which can degrade performance and compromise safety. Detecting such substitutions is difficult, as users lack access to model weights and, in most cases, even output logits. To tackle this problem, we propose a rank-based uniformity test that can verify the behavioral equality of a black-box LLM to a locally deployed authentic model. Our method is accurate, query-efficient, and avoids detectable query patterns, making it robust to adversarial providers that reroute or mix responses upon the detection of testing attempts. We evaluate the approach across diverse threat scenarios, including quantization, harmful fine-tuning, jailbreak prompts, and full model substitution, showing that it consistently achieves superior statistical power over prior methods under constrained query budgets.

1 Introduction

APIs have become a central access point for large language models (LLMs) in consumer applications, enterprise tools, and research workflows (Anysphere Inc., 2025; Yun et al., 2025; ResearchFlow, 2025). However, while users can query black-box APIs, they have little to no visibility into the underlying model implementation. Combined with the high cost of serving large models and the latency pressure to reduce time-to-first-token (TTFT), API providers are incentivized to deploy smaller or quantized variants of the original model to cut costs. Such modifications, while opaque to end users, can degrade model performance and introduce safety risks (Egashira et al., 2024). In more concerning cases, providers may incorporate harmful fine-tuning, jailbreak-enabling system prompts, or even misconfigured system components without realizing it (mirpo, 2025).

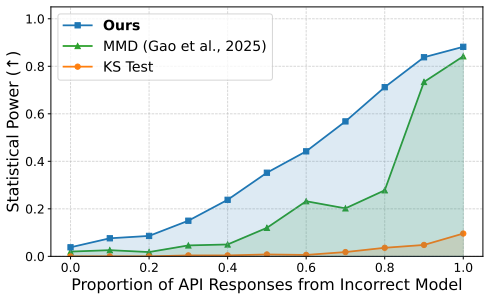

Figure 1: Statistical power of different methods in detecting substitution of the Gemma-2-9b-it with its 4-bit quantized variant, as the proportion of API responses from the quantized model increases. Our method significantly outperforms MMD (Gao et al., 2025) and the Kolmogorov–Smirnov (KS) baseline.

Preprint. Under review.

These risks highlight the need for *LLM API auditing*—the task of checking whether a deployed model is as claimed. Yet this is particularly challenging in the black-box setting: users typically lack access to model weights and receive only limited metadata (e.g., top-5 token log-probabilities). This necessitates detection methods that rely solely on observed outputs. However, even such output-level methods face potential evasion: if the detection relies on invoking an LLM API with specially constructed query distributions, a dishonest API provider could detect the special pattern and reroute those queries to the original model they claim to serve. Worse still, even without knowing the detection strategy, an API provider could mix multiple models, making the response distribution harder to distinguish.

Following Gao et al. (2025), we formulate the LLM API auditing problem as a *model equality test*: given query access to a target LLM API and a certified reference model of the expected configurations, the goal is to determine if the two produce statistically indistinguishable outputs on shared prompts.

We propose that a model equality test for auditing LLM APIs must satisfy three key criteria: *accuracy*, *query efficiency*, and *robustness* to adversarial attacks. Accuracy reflects how reliably a test can be used in practice. Query efficiency is critical for reducing operational overhead, which incentivizes more audits to ensure API models' integrity. Robustness is equally essential for real-world deployment, where audits must both evade detection by adversarial API providers and remain effective under targeted attacks.

While several methods have been proposed for model equality testing, they each fall short in one or more of these criteria (Table 1). Existing methods include Maximum Mean Discrepancy (MMD) (Gao et al., 2025), trained text classifiers (Sun et al., 2025), identity prompting (Huang et al., 2025), and benchmark performance comparison (Chen et al., 2023a). However, Sun et al. (2025) require prohibitively many API queries; Huang et al. (2025) fail to capture model variations such as size, version, or quantization (Cai et al., 2025); and Gao et al. (2025) and Chen et al. (2023a) rely on special query distributions that can be adversarially detected and circumvented by techniques such as prompt caching (Gu et al., 2025).

Driven by these limitations, we propose a rank-based uniformity test (RUT)—an asymmetric two-sample hypothesis test that addresses all three criteria simultaneously. In RUT, we sample one response from the target API and multiple responses from the reference model for each prompt, then compute the rank percentile of the API output within the reference distribution. If the target and reference models are identical, the percentiles should follow a uniform distribution. We detect deviations using the Cramér–von Mises test (Cramér, 1928). Our method requires only a single API call per prompt, operates effectively on real-world, user-like queries, and avoids detectable patterns that adversarial providers might exploit.

Table 1: Comparison of LLM auditing methods by *accuracy* (Acc.), *query-efficiency* (Q-Eff.), and *robustness to adversarial providers* (Rob.).

Method	Acc.	Q-Eff.	Rob.
RUT (Ours)	✓	✓	✓
MMD (Gao et al., 2025)	✓	✓	✗
Classifier (Sun et al., 2025)	✗	✗	✓
Identity-prompting (Huang et al., 2025)	✗	✓	✗
Benchmark (Chen et al., 2023a)	✗	✗	✗

We evaluate RUT across a range of adversarial scenarios in which the API provider secretly substitutes the claimed model with an alternative. In Section 5.2, we study the case where the substitute is a quantized version of the original model. In Section 5.3, we test models augmented with a hidden jailbreaking system prompt. In Section 5.4, we examine models finetuned on instruction-following data. Finally, in Section 5.5, we consider substitution with a completely different model.

Under a fixed API query budget, we find RUT outperforms both MMD and a Kolmogorov–Smirnov test (KS) baseline across all settings. It consistently achieves higher statistical power and shows greater robustness to probabilistic substitution attacks (Figure 1). Moreover, when applied to five real-world API-deployed models (Section 5.6), our method yields detection results closely aligned with other methods and shows more robustness over string-based metrics on minor decoding mismatches.

To summarize, the main contributions of our work include:

1. **A novel statistically-principled test for auditing LLM APIs.** We propose RUT, an asymmetric two-sample-test that needs only one API call per prompt and operates effectively on natural queries, achieving query efficiency and by-deign robustness to adversarial providers.

2. **Empirical validation across diverse threat models.** We perform comprehensive experiments to validate RUT's superior statistical power under diverse model substitution settings,

including quantization, jailbreaking, supervised finetuning (SFT), full model replacement, and hardware/provider replacement.

3. **Cross-validated audit of live commercial endpoints.** We benchmark RUT side-by-side with established tests (MMD and KS) on three major public LLM APIs and demonstrate its practicality in real-world black-box settings.

2 Related Work

LLM fingerprinting. Fingerprinting approaches focus on identifying LLMs by analyzing their outputs. Active fingerprinting involves injecting backdoor-like behavior (Xu et al., 2024) into an LLM via finetuning, embedding watermarks (Kirchenbauer et al., 2023; Ren et al., 2023) into a model' s text generation process, or intentionally crafting prompts to elicit unique outputs from different LLMs (Pasquini et al., 2024). Passive fingerprinting, on the other hand, focuses on analyzing the inherent patterns in LLM-generated text (Su et al., 2023; Fu et al., 2025; Alhazbi et al., 2025). This builds on the observation that LLMs expose rich "idiosyncrasies"—distributional quirks that allow classifiers to identify a model (Sun et al., 2025). While passive fingerprinting is relevant for LLM auditing, many such methods rely on training classifiers and require substantial labeled data, making them suboptimal for auditing LLM APIs. Prior work (Cai et al., 2025) also shows they are ineffective in detecting quantized model substitution.

Auditing LLM APIs. A growing body of work investigates whether black-box APIs faithfully serve the advertised model. The most straightforward audit is to evaluate models' benchmark performance (Analysis, 2025; Eyuboglu et al., 2024; Chen et al., 2023b), but raw performance alone cannot expose covert substitutions or partial routing. Gao et al. (2025) formalizes the problem as *Model Equality Testing* and shows that a kernel-MMD test can already flag public endpoints that deviate from their open-weight checkpoints. Concurrently to our work, Cai et al. (2025) investigate the model substitution setting and show that API providers can evade detection through strategies such as model quantization, randomized substitution, and benchmark evasion. Building on these insights, we propose a method that is more robust to such attacks and extend the threat model to include a broader range of realistic scenarios, such as jailbroken or maliciously finetuned models.

3 Problem Formulation

In this section, we formalize the LLM API auditing problem as a black-box model equality test between a target API and a fully accessible reference model.

Models. We denote an LLM as a conditional distribution $\pi(y|x;\varphi)$ over text output $y \in \mathcal{Y}$ given an input prompt $x \in \mathcal{X}$ and decoding parameters φ (e.g., temperature, top-p). In all experiments, we control the decoding parameters to be the same across models. Two such models are involved in the test. The *reference model* π_{ref} is fully accessible locally; the verifier can draw an unlimited number of samples $y_{\text{ref}} \sim \pi_{\text{ref}}(\cdot \mid x;\varphi)$ for any x and obtain relevant logits. The *target model* π_{tgt} is exposed only through a remote API that returns exactly one completion $y_{\text{tgt}} \sim \pi_{\text{tgt}}(\cdot \mid x;\varphi)$ per call and charges for every invocation. The total API budget is N prompts.

Hypotheses. The model equality test is cast as a composite two-sample hypothesis test. The null hypothesis

$$H_0: \quad \pi_{\text{tgt}}(\cdot \mid x;\varphi) \;=\; \pi_{\text{ref}}(\cdot \mid x;\varphi) \quad \forall\, x \in \mathcal{X} \tag{1}$$

states that the deployed model is behaviorally identical to the declared one.

$$H_1: \quad \exists\, x \in \mathcal{X} \text{ such that } \pi_{\text{tgt}}(\cdot \mid x;\varphi) \neq \pi_{\text{ref}}(\cdot \mid x;\varphi) \tag{2}$$

captures any observable deviation, whether arising from quantization, fine-tuning, prompt injection, or full model replacement.

Adversarial model provider. A dishonest model provider may attempt to evade detection through *probabilistic substitution*. We model this behaviour by

$$\pi_{\text{tgt}}(\cdot \mid x;\varphi) \;=\; \big(1 - q(x)\big)\,\pi_{\text{ref}}(\cdot \mid x;\varphi) \;+\; q(x)\,\pi_{\text{alt}}(\cdot \mid x;\varphi), \tag{3}$$

where π_{alt} is an arbitrary alternative model (e.g., quantized or adversarially finetuned version of π_{ref}) and $q : \mathcal{X} \to [0,1]$ is an unknown routing function that may depend on the prompt and even on past